FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Ken Dylen R. Fernando & Nadine Love Ramos
Section:	AN43
Professor:	Crisola Tan
Date:	October 31, 2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.
Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking
✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ "New case saved" or similar confirmation
✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System
A CBR system identifies the learning struggles of students. Student submits a math solution with repeated errors. System retrieves past students with similar error patterns (e.g., misusing formulas). It adapts feedback and personalized lessons based on successful interventions. **Learning:** System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

Case Study 3 Intelligent Tutoring System

Overview

This project is a simple Intelligent Tutoring System developed in Python. It uses Case-Based Reasoning to recommend a lesson for a learner who makes a particular math error. The program represents previous learning situations as cases, retrieves a similar case, adapts its lesson, and saves a reviewed result for future use. The demonstration focuses on adding fractions with unlike denominators. All learner profiles and outcome data are fictional examples used to show how the reasoning process works.

Case representation

The initial case base contains eight cases stored in seed_cases.json. Each case has an ID, a problem description, and a solution. The problem includes the math topic, error type, mastery score from 0 to 100, preferred lesson format, and available time. The solution contains feedback, an exercise count, a lesson format, and a duration. These fields describe both the learning difficulty and the intervention that can be reused.

Three cases concern unlike denominators, one concerns confusion between the numerator and denominator, two concern linear equations, and two concern geometry. For example, C02 describes a learner studying fractions with unlike denominators, mastery 60, and a preference for worked examples. Its lesson gives a concrete fraction example and five exercises over 20 minutes. Having several cases for the same error allows the program to distinguish between learners with different mastery levels and lesson preferences.

Similarity assessment and retrieval

The program calculates a weighted similarity score for every case. A matching topic contributes 30%, a matching error contributes 35%, mastery similarity contributes 20%, and a matching lesson format contributes 15%. Category comparisons receive 1 for a match and 0 for a mismatch. Mastery similarity is

calculated as 1 minus the absolute difference between the two scores divided by 100. The total score ranges from 0 to 1, with a higher score indicating a closer match.

Similarity = 0.30 × topic match + 0.35 × error match + 0.20 × mastery similarity + 0.15 × format match.

The error type has the greatest weight because the feedback must address the learner's actual mistake. Available time is handled during adaptation rather than retrieval. The ranked results are sorted from highest to lowest, with the case ID used to break a tie. For the demonstration learner, C02 receives 0.960: 0.30 + 0.35 + 0.20(0.80) + 0.15. It is therefore retrieved as the most similar past case.

Case adaptation and review

The new learner has mastery 40, prefers a worked example, and has 15 minutes. The program reuses C02's feedback because the topic and error match. For mastery below 50, it adds two exercises and provides guided steps with a check after each step. For mastery of 50 or above, it uses independent practice with a final check. A case with a different topic or error is rejected during adaptation.

The lesson must also fit the available time. The program reserves three minutes for explanation and allows two minutes per exercise. Although the lower mastery rule increases five exercises to seven, the 15-minute limit allows only six. The final lesson therefore contains six exercises, guided checks, a worked example, and a 15-minute duration. The feedback explains 1/2 + 1/3 = 3/6 + 2/6 = 5/6 and asks why the denominator stays 6.

Review is represented by recording an outcome and an improved mastery score before retention. The prototype does not automatically grade a student's work or prove that an intervention is effective. In the demonstration, the reviewed outcome is that the fictional learner corrected four of five items and improved from mastery 40 to 60.

Learning and retention

After a successful review, the program stores the new problem, adapted lesson, source case ID, outcome, and review ID in cases.json. On the first demonstration save, it adds C09 and increases the case base from eight to nine cases. A temporary file is written before replacing the saved file. The new case remains available when the program is restarted. A review ID prevents the same demonstration result from being added repeatedly. This is the learning stage of CBR: the system gains another experience that can be compared with later problems.

Running the Python program

The program uses only Python's standard library. Keep cbr.py and seed_cases.json in the same folder and open a terminal in that folder. Run python cbr.py to display retrieval and adaptation without adding a case. Run python cbr.py --save-demo to also retain the fictional reviewed outcome. The first run starts with eight seed cases if cases.json does not exist. Later runs load the saved case base, so rankings can change when a new, closer case has been retained.

Results and limitations

A fresh demonstration ranks the cases as C02 0.960, C01 0.840, C08 0.770, C03 0.480, C06 0.350, C04 0.340, C05 0.170, and C07 0.150. The retrieved case, adapted lesson, save confirmation, and updated

case list are shown in the execution output below. Checks on the implementation confirmed the ranking, lesson time limit, persistence, and duplicate-save prevention.

The system is a small prototype with a fixed set of topics, manually chosen weights, and a command-line demonstration input. It does not analyze handwritten answers or replace teacher judgment. Its recommendations depend on the quality of its stored cases. Within this scope, it demonstrates how experience can be represented and reused through the CBR cycle.

Pair reflection

Our project shows how a learner's topic, error, mastery, and lesson preference can be represented as a reusable case. The programming side involves defining the data, calculating similarity, adapting the lesson, and saving reviewed results. The documentation and checking side involves explaining those decisions and comparing the outputs with the intended behavior. A central design challenge is choosing a case that addresses the same mistake while still considering differences between learners. Another challenge is providing enough practice without exceeding the available lesson time. The weighted score and exercise limit make these decisions clear and easy to inspect. The main lesson from the project is that CBR depends on relevant past cases, careful adaptation, and reviewed outcomes that improve the case base.

Python execution output

Captured text from a fresh run of python cbr.py --save-demo. The saved outcome is fictional.

```
New problem: {'topic': 'Fractions', 'error': 'Unlike denominators', 'mastery': 40, 'format':
'Worked example', 'minutes': 15}
Similarity ranking:
C02 0.96
C01 0.84
C08 0.77
C03 0.48
C06 0.35
C04 0.34
C05 0.17
C07 0.15
Retrieved case: C02
Adapted lesson: {
   "feedback": "Work through 1/2 + 1/3 = 3/6 + 2/6 = 5/6; explain why the denominator stays
6.",
  "format": "Worked example",
  "exercises": 6,
  "minutes": 15,
  "support": "Guided steps with a check after each step"
}
New case saved: C09
Case base: ['C01', 'C02', 'C03', 'C04', 'C05', 'C06', 'C07', 'C08', 'C09']
```

Repository Link:

https://github.com/krfernando-FEUA/Fernando-Ken-Dylen-CS0065-AN43/tree/main/Technical-Assessment-03

Python source code

File: cbr.py. The starting cases are loaded from the accompanying seed_cases.json file.

```
import argparse
import json
from pathlib import Path

ROOT = Path(__file__).resolve().parent
CASE_FILE = ROOT / "cases.json"
TOPICS = {
   "Fractions": ["Unlike denominators", "Numerator and denominator confusion"],
   "Linear equations": ["Sign error", "Unbalanced operations"],
   "Geometry": ["Wrong formula", "Unit confusion"],
}
FORMATS = ["Visual", "Worked example", "Practice"]
DEMO = {"topic": "Fractions", "error": "Unlike denominators", "mastery": 40,
       "format": "Worked example", "minutes": 15}


def load_cases(path=CASE_FILE):
   if not path.exists():
       path = ROOT / "seed_cases.json"
   return json.loads(path.read_text(encoding="utf-8"))


def validate(problem):
   topic = problem.get("topic")
   if topic not in TOPICS or problem.get("error") not in TOPICS[topic]:
       raise ValueError("Select a topic and a matching error.")
```

```
    if problem.get("format") not in FORMATS:
        raise ValueError("Select a supported lesson format.")
    for key, low, high in [("mastery", 0, 100), ("minutes", 5, 60)]:
        value = problem.get(key)
        if type(value) is not int or not low <= value <= high:
            raise ValueError(f"{key} must be a whole number from {low} to
{high}.")

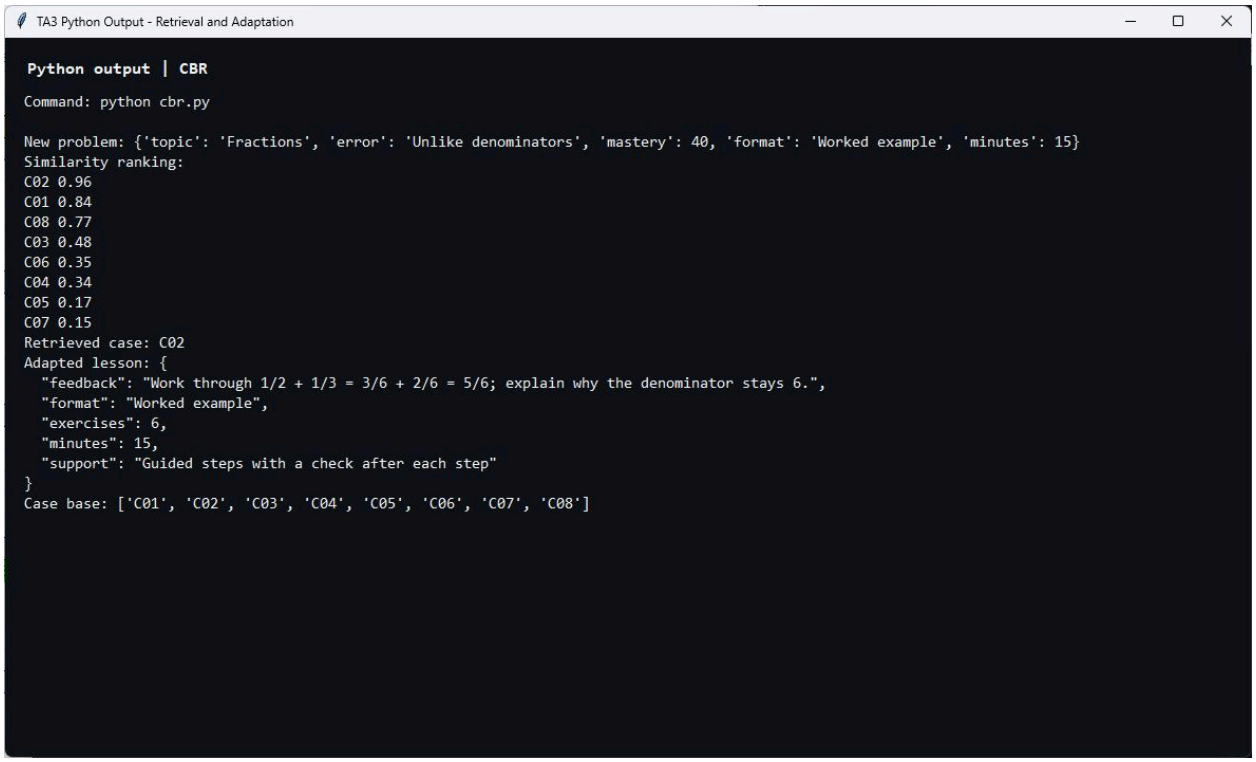

def retrieve(problem, cases):
    """Compare each past case and return the highest scores first."""
    validate(problem)
    results = []
    for case in cases:
        past = case["problem"]
        score = (0.30 * (problem["topic"] == past["topic"])
                 + 0.35 * (problem["error"] == past["error"])
                 + 0.20 * (1 - abs(problem["mastery"] - past["mastery"]) / 100)
                 + 0.15 * (problem["format"] == past["format"]))
        results.append({"case": case, "score": score})
    return sorted(results, key=lambda item: (-item["score"],
item["case"]["id"]))

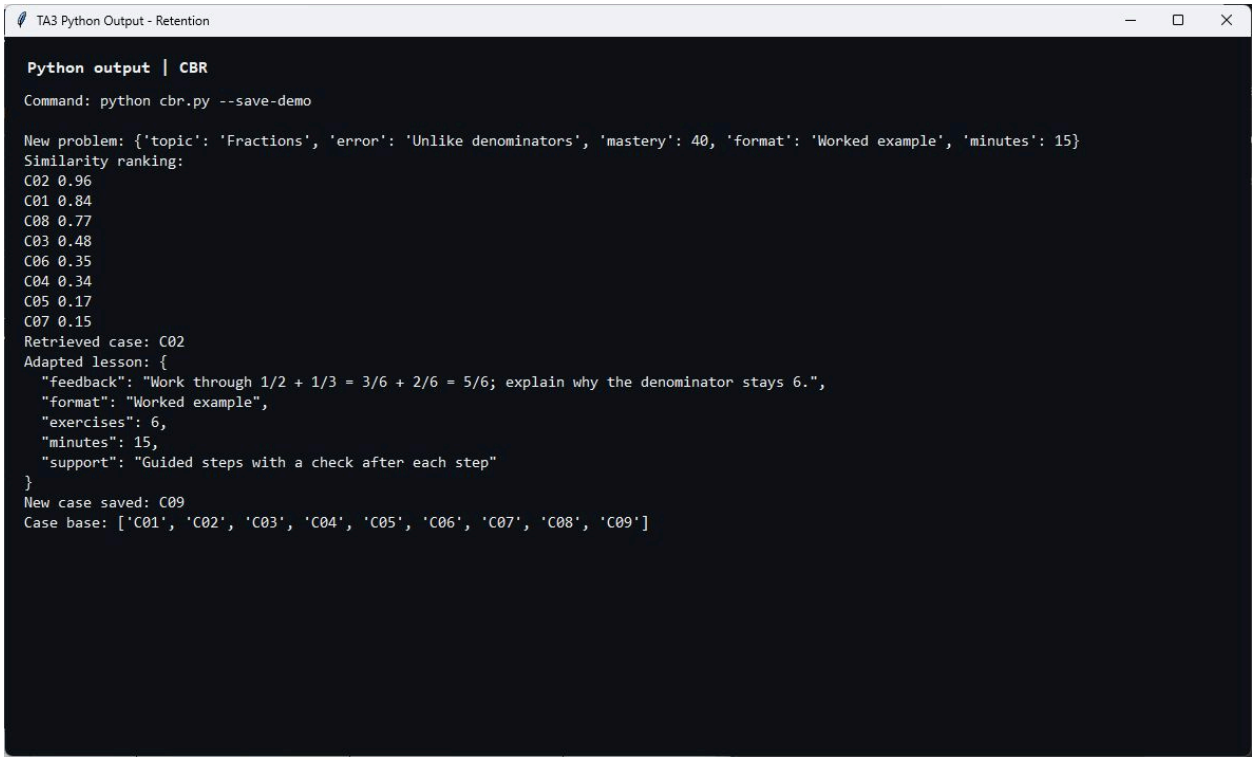

def adapt(problem, case):
    """Reuse feedback and adjust the lesson to the learner's needs."""
    validate(problem)
    if any(problem[key] != case["problem"][key] for key in ["topic", "error"]):
        raise ValueError("No matching lesson. Ask a teacher to add a case.")
    solution = case["solution"].copy()
    exercises = solution["exercises"]
    if problem["mastery"] < 50:
        exercises += 2
        solution["support"] = "Guided steps with a check after each step"
    else:
```

```
        solution["support"] = "Independent practice with a final check"
    time_limit = max(1, (problem["minutes"] - 3) // 2)
    solution["exercises"] = min(exercises, time_limit)
    solution["format"] = problem["format"]
    solution["minutes"] = problem["minutes"]
    return solution


def retain(problem, solution, source_id, outcome, post_mastery, review_id,
path=CASE_FILE):
    """Save a reviewed successful lesson. A review ID prevents duplicate
saves."""
    validate(problem)
    if not solution["feedback"].strip() or not outcome.strip() or not
review_id.strip():
        raise ValueError("Enter reviewed feedback, an outcome and a review ID.")
    if type(post_mastery) is not int or not problem["mastery"] < post_mastery <=
100:
        raise ValueError("Post-lesson mastery must improve and be at most 100.")
    cases = load_cases(path)
    for case in cases:
        if case.get("review_id") == review_id:
            return case
    if source_id not in [case["id"] for case in cases]:
        raise ValueError("The source case is missing.")
    new_case = {"id": f"C{len(cases) + 1:02}", "problem": problem.copy(),
                "solution": solution.copy(), "source_case_id": source_id,
                "outcome": {"notes": outcome.strip(), "post_mastery":
post_mastery},
                "review_id": review_id}
    cases.append(new_case)
    temporary = path.with_suffix(".tmp")
    temporary.write_text(json.dumps(cases, indent=2), encoding="utf-8")
    temporary.replace(path)
    return new_case
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--save-demo", action="store_true")
    args = parser.parse_args()
    print("New problem:", DEMO)
    ranking = retrieve(DEMO, load_cases())
    print("Similarity ranking:")
    for item in ranking:
        print(item["case"]["id"], round(item["score"], 3))
    best = ranking[0]["case"]
    solution = adapt(DEMO, best)
    print("Retrieved case:", best["id"])
    print("Adapted lesson:", json.dumps(solution, indent=2))
    if args.save_demo:
        saved = retain(DEMO, solution, best["id"],
                       "Fictional demo: learner corrected 4 of 5 items.", 60,
"cli-demo")
        print("New case saved:", saved["id"])
    print("Case base:", [case["id"] for case in load_cases()])
```


TA3 Python Output - Retrieval and Adaptation
Python output | CBR
Command: python cbr.py
New problem: {'topic': 'Fractions', 'error': 'Unlike denominators', 'mastery': 40, 'format': 'Worked example', 'minutes': 15}
Similarity ranking:
C02 0.96
C01 0.84
C08 0.77
C03 0.48
C06 0.35
C04 0.34
C05 0.17
C07 0.15
Retrieved case: C02
Adapted lesson: {
"feedback": "Work through 1/2 + 1/3 = 3/6 + 2/6 = 5/6; explain why the denominator stays 6.",
"format": "Worked example",
"exercises": 6,
"minutes": 15,
"support": "Guided steps with a check after each step"
}
Case base: ['C01', 'C02', 'C03', 'C04', 'C05', 'C06', 'C07', 'C08']


TA3 Python Output - Retention
Python output | CBR
Command: python cbr.py --save-demo
New problem: {'topic': 'Fractions', 'error': 'Unlike denominators', 'mastery': 40, 'format': 'Worked example', 'minutes': 15}
Similarity ranking:
C02 0.96
C01 0.84
C08 0.77
C03 0.48
C06 0.35
C04 0.34
C05 0.17
C07 0.15
Retrieved case: C02
Adapted lesson: {
"feedback": "Work through 1/2 + 1/3 = 3/6 + 2/6 = 5/6; explain why the denominator stays 6.",
"format": "Worked example",
"exercises": 6,
"minutes": 15,
"support": "Guided steps with a check after each step"
}
New case saved: C09
Case base: ['C01', 'C02', 'C03', 'C04', 'C05', 'C06', 'C07', 'C08', 'C09']